RateKing

User Guide

Pedro H. F. Matias

Version 0.1.0 - February 2026

Contents

1 Introduction **5**
- 1.1 Key Features . . . 5
- 1.2 Requirements . . . 5

2 Installation **7**
- 2.1 Standard Installation . . . 7
- 2.2 Development Installation . . . 7
- 2.3 Verification . . . 7

3 Theory **9**
- 3.1 Transition State Theory . . . 9
- 3.2 Partition Functions . . . 9
  - 3.2.1 Vibrational . . . 9
  - 3.2.2 Rotational . . . 9
  - 3.2.3 Translational . . . 9
- 3.3 Tunneling Corrections . . . 10
  - 3.3.1 Wigner Correction . . . 10
  - 3.3.2 Bell Corrections . . . 10
  - 3.3.3 Eckart Tunneling . . . 10

4 API Reference **11**
- 4.1 Core Classes . . . 11
  - 4.1.1 Molecule . . . 11
  - 4.1.2 TransitionState . . . 11
  - 4.1.3 Reaction . . . 11
- 4.2 Rate Constant Calculators . . . 12
  - 4.2.1 TST . . . 12
  - 4.2.2 dTST - Deformed Transition State Theory . . . 12
  - 4.2.3 Tunneling Corrections . . . 12
- 4.3 Utility Classes . . . 13
  - 4.3.1 CONSTANTS . . . 13

5 Practical Examples **15**
- 5.1 Example 1: Single Molecule . . . 15
- 5.2 Example 2: Complete Reaction . . . 15
- 5.3 Example 3: Temperature Range . . . 16
- 5.4 Example 4: Check Available Links . . . 16

6 Troubleshooting **17**
6.1 Common Errors . . . 17
6.1.1 Import Error . . . 17
6.1.2 File Not Found . . . 17
6.1.3 No Imaginary Frequency . . . 17
6.1.4 Zero Partition Function . . . 18
6.2 Validation Checks . . . 18

7 Best Practices **19**
7.1 Gaussian Input Files . . . 19
7.2 Project Organization . . . 19
7.3 Reusable Functions . . . 19

A Constants **21**

B References **23**

Chapter 1

Introduction

RateKing is a Python package for calculating chemical reaction rate constants using Transition State Theory (TST) and quantum mechanical tunneling corrections.

1.1 Key Features

- Transition State Theory (TST) calculations
- Deformed Transition State Theory
- Tunneling corrections: Eckart, Wigner, Bell (1935/1958/2T)
- Automatic extraction from Gaussian 16 output files
- Multi-link file support
- Temperature-dependent calculations

1.2 Requirements

- Python >= 3.10
- MoleKing package
- NumPy and SciPy

Chapter 2

Installation

2.1 Standard Installation

```
pip install RateKing
```

2.2 Development Installation

```
git clone https://github.com/phfmatias/RateKing.git
cd RateKing
pip install -e .
```

2.3 Verification

```
import RateKing
print(RateKing.__version__)
from RateKing import Molecule, Reaction, TST
```

Chapter 3

Theory

3.1 Transition State Theory

The TST rate constant is given by:

$$k_{\mathrm{TST}}(T) = \frac{k_B T}{h} \frac{Q^{\ddagger}}{Q_R} \exp\left(-\frac{E_a}{RT}\right) \tag{3.1}$$

where:

- k_B = Boltzmann constant
- h = Planck constant
- $Q^{\ddagger}$ = transition state partition function
- Q_R = reactants partition function
- E_a = activation energy

3.2 Partition Functions

The total partition function is:

$$Q = q_{\mathrm{elec}} \cdot q_{\mathrm{vib}} \cdot q_{\mathrm{rot}} \cdot q_{\mathrm{trans}} \tag{3.2}$$

3.2.1 Vibrational

$$q_{\mathrm{vib}} = \prod_i \frac{e^{-h\nu_i/(2k_BT)}}{1 - e^{-h\nu_i/(k_BT)}} \tag{3.3}$$

3.2.2 Rotational

$$q_{\mathrm{rot}} = \frac{8\pi^2 I k_B T}{\sigma h^2} \tag{3.4}$$

3.2.3 Translational

$$q_{\mathrm{trans}} = \left(\frac{2\pi m k_B T}{h^2}\right)^{3/2} V \tag{3.5}$$

3.3 Tunneling Corrections

3.3.1 Wigner Correction

$$\kappa_W = 1 + \frac{1}{24}\left(\frac{h|\nu_i|c}{k_B T}\right)^2 \tag{3.6}$$

3.3.2 Bell Corrections

The Bell (1935) correction accounts for tunneling through barrier:

$$\kappa_{B35} = \frac{k_B T - \hbar|\nu_i|ce^{-\alpha}}{k_B T - \hbar|\nu_i|c} \tag{3.7}$$

The Bell (1958) correction:

$$\kappa_{B58} = \frac{\beta}{\sin\beta}, \quad \beta = \frac{h|\nu_i|c}{2k_B T} \tag{3.8}$$

3.3.3 Eckart Tunneling

The Eckart correction involves numerical integration:

$$\kappa_E = \frac{1}{RT}\int_0^\infty P(E)\exp\left(-\frac{E - E_a}{RT}\right)dE \tag{3.9}$$

where $P(E)$ is the energy-dependent transmission coefficient.

Chapter 4

API Reference

4.1 Core Classes

4.1.1 Molecule

```
from RateKing import Molecule

molecule = Molecule(
    name="NH3",
    filePath="path/to/file.out",
    link=-1,
    temperature=298.15
)
```

Attributes:

- `zeroPointEnergy`: ZPE in Hartree
- `enthalpy`: Enthalpy in Hartree
- `entropy`: Entropy in cal/(mol K)
- `gibbs`: Gibbs free energy
- `qTotal`: Total partition function
- `symmetryNumber`: Molecular symmetry number

4.1.2 TransitionState

Inherits from Molecule with imaginary frequency:

```
from RateKing import TransitionState

ts = TransitionState("TS", "path/to/ts.out")
print(f"Imaginary freq: {ts.imaginaryFrequency} cm^-1")
```

4.1.3 Reaction

```
from RateKing import Reaction

reaction = Reaction(
    name="Reaction␣name",
    reactantsPath=[
        {"name": "R1", "filePath": "r1.out", "link": -1},
        {"name": "R2", "filePath": "r2.out", "link": -1}
    ],
    productsPath=[
        {"name": "P1", "filePath": "p1.out", "link": -1},
        {"name": "P2", "filePath": "p2.out", "link": -1}
    ],
    transitionStatePath={
        "name": "TS", "filePath": "ts.out", "link": -1
    },
    temperature=298.15
)
reaction.run()
```

Important attributes after run():

- `activationEnergy`: Ea in kcal/mol
- `activationEnergy_cal`: Ea in cal/mol
- `deltaE`, `deltaH`, `deltaG`: Thermochemistry
- `reactionTotalPartitionFunction`: Q ratio

4.2 Rate Constant Calculators

4.2.1 TST

```
from RateKing import TST

tst = TST(reaction)
k_tst = tst.calculateKTST(bi_or_uni='bi')
print(f"k_TST␣=␣{k_tst:.3e}␣cm^3/molecule/s")
```

4.2.2 dTST - Deformed Transition State Theory

```
from RateKing import dTST

dtst = dTST(reaction)
k_dtst = dtst.calculateKDTST(bi_or_uni='bi')
```

4.2.3 Tunneling Corrections

```
from RateKing.tunneling_corrections import (
    eckartCorrection,
    wignerCorrection,
    bell35Correction
```

```
)

eckart = eckartCorrection(reaction)
k_eckart = k_tst * eckart.kappaEckart

wigner = wignerCorrection(reaction)
k_wigner = k_tst * wigner.kappaWigner
```

4.3 Utility Classes

4.3.1 CONSTANTS

```
from RateKing import CONSTANTS

const = CONSTANTS()
const.load_constants()
print(f"k_B = {const.k_B} J/K")
print(f"h = {const.h} J*s")
print(f"NA = {const.NA} mol^-1")
```

Chapter 5

Practical Examples

5.1 Example 1: Single Molecule

```
from RateKing import Molecule

mol = Molecule("NH3", "nh3.out", temperature=298.15)
print(f"ZPE: {mol.zeroPointEnergy} Hartree")
print(f"Partition function: {mol.qTotal}")
```

5.2 Example 2: Complete Reaction

```
from RateKing import Reaction, TST
from RateKing.tunneling_corrections import eckartCorrection

rxn = Reaction(
    name="NH3+OH",
    reactantsPath=[
        {"name": "NH3", "filePath": "nh3.out", "link": 2},
        {"name": "OH", "filePath": "oh.out", "link": 2}
    ],
    productsPath=[
        {"name": "NH2", "filePath": "nh2.out", "link": 2},
        {"name": "H2O", "filePath": "h2o.out", "link": 2}
    ],
    transitionStatePath={
        "name": "TS", "filePath": "ts.out", "link": 2
    },
    temperature=298.15
)

rxn.run()
print(f"Ea = {rxn.activationEnergy:.2f} kcal/mol")

tst = TST(rxn)
k = tst.calculateKTST()
print(f"k_TST = {k:.3e}")

eckart = eckartCorrection(rxn)
k_corr = k * eckart.kappaEckart
```

```
print(f"k_Eckart = {k_corr:.3e}")
```

5.3 Example 3: Temperature Range

```
import numpy as np

temps = np.linspace(200, 500, 7)

for T in temps:
    rxn = Reaction(..., temperature=T)
    rxn.run()
    k = TST(rxn).calculateKTST()
    print(f"{T:6.1f} K: {k:.3e}")
```

5.4 Example 4: Check Available Links

```
from RateKing import Molecule

total = Molecule.getAvailableLinks("file.out")
print(f"File has {total} links")

mol = Molecule("Sp", "file.out", link=2)
```

Chapter 6

Troubleshooting

6.1 Common Errors

6.1.1 Import Error

Problem: `ModuleNotFoundError: No module named 'RateKing'`

Solution:

```
pip install RateKing
pip install --upgrade RateKing  # If needed
```

6.1.2 File Not Found

Problem: File path errors

Solution: Use absolute paths:

```
import os
path = os.path.abspath("file.out")
print(path)
```

6.1.3 No Imaginary Frequency

Problem: `ExtractionError: No imaginary frequency found`

Causes:

- File is not a transition state
- Frequency calculation incomplete
- Wrong link number

Solution:

1. Verify TS optimization in Gaussian
2. Check frequency calculation finished
3. Try different link: `link=1`, `link=2`, etc.

6.1.4 Zero Partition Function

Problem: `CalculationError: Zero partition function`
Solution:

- Check Gaussian output is complete
- Verify thermochemistry section exists
- Use reasonable temperature (not near 0 K)

6.2 Validation Checks

```
# Check reasonable values
rxn.run()
print(f"Ea = {rxn.activationEnergy:.1f} kcal/mol")
if rxn.activationEnergy < 0:
    print("WARNING: Negative activation energy!")

imag_freq = abs(rxn.transitionStateImaginaryFrequency)
if imag_freq < 100 or imag_freq > 2000:
    print("WARNING: Unusual imaginary frequency")
```

Chapter 7

Best Practices

7.1 Gaussian Input Files

For best results:

```
%NProcShared=8
%Mem=16GB
#P M062X/6-311++G(d,p) Opt=(TS,CalcFC) Freq Temperature=298.15

Title

0 1
[geometry]
```

7.2 Project Organization

```
project/
  data/
    reactants/
      r1.out
      r2.out
    products/
      p1.out
      p2.out
    ts/
      ts.out
  scripts/
    calculate.py
    plot.py
  results/
    rates.csv
```

7.3 Reusable Functions

```
def calculate_reaction(T):
    "Calculate rate at temperature T"
    rxn = Reaction(..., temperature=T)
    rxn.run()
```

```
    k_tst = TST(rxn).calculateKTST()
    k_eckart = k_tst * eckartCorrection(rxn).kappaEckart
    return {'T': T, 'k_tst': k_tst, 'k_eckart': k_eckart}

# Usage
import numpy as np
temps = np.arange(250, 501, 50)
for T in temps:
    result = calculate_reaction(T)
    print(f"{result['T']} K: {result['k_tst']:.3e}")
```

Appendix A

Constants

Constant	**Value**	**Units**
k_B	1.380649e-23	J/K
h	6.62607015e-34	J*s
c (speed of light)	2.99792458e10	cm/s
R	1.987204	cal/(mol*K)
N_A	6.02214076e23	1/mol
P_0	101325	Pa

Appendix B

References

1. Eyring, H. (1935). The Activated Complex in Chemical Reactions. J. Chem. Phys., 3, 107-115.
2. Evans, M. G.; Polanyi, M. (1935). Applications of transition state method. Trans. Faraday Soc., 31.
3. Wigner, E. (1932). Theory of barrier crossing. Z. Phys. Chem. B, 19, 203-216.
4. Bell, R. P. (1980). The Tunnel Effect in Chemistry. Chapman & Hall.
5. Truhlar, D. G.; Garrett, B. C. (1984). Variational Transition State Theory. Annu. Rev. Phys. Chem., 35.
6. Frisch, M. J. et al. (2016). Gaussian 16 Rev. C.01. Gaussian, Inc.